

Clipnote: AI Lecture Note-Taker & Study App

■ LIVE SECURE APPLICATION URL (HTTPS):
<https://palace-subjective-except-stats.trycloudflare.com/>
(Direct AWS EC2 IP: http://3.27.186.233)

Application Name:	Clipnote	Live HTTPS URL:	https://palace-subjective-except-stats.trycloudflare.com/
Primary LLM:	Google Gemini 2.5 Flash API	Cloud Platform:	AWS EC2 (Ubuntu 24.04 / t3.micro)
Frontend Stack:	Next.js 16 (App Router), Tailwind v4	Backend Stack:	FastAPI, SQLite, Nginx, Systemd, PM2

1. Application Overview & Tech Stack

Clipnote is a full-stack, AI-powered lecture processing application that ingests recorded audio/video files or YouTube links and automatically synthesizes comprehensive study materials. The system generates structured sectioned notes, key concepts glossaries, interactive 5-question multiple-choice quizzes, executive revision guides, and multi-format exports (PDF, Markdown, Anki flashcard decks).

Layer	Technology / Framework	Key Purpose & Responsibility
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, Lucide Icons	Responsive dark-mode UI, real-time status polling, tabbed dashboard, synchronized media player.
Backend API	FastAPI, Uvicorn, Python 3.12, Pydantic v2, SQLAlchemy	RESTful endpoints, background task processing queue, non-blocking asynchronous pipeline execution.
Database	SQLite 3 with SQLAlchemy ORM	Persistent storage of jobs, raw transcripts, timestamped segments, and generated JSON study kits.
AI / LLM Engine	Google Gemini 2.5 Flash (`gemini-2.5-flash`), OpenAI GPT-4o-mini	Primary structural synthesis, JSON schema enforcement, multi-segment transcript summarization.
Media & Ingestion	youtube-transcript-api, pytubefix, yt-dlp, FFmpeg, ReportLab	Audio extraction, VTT/SRT caption parsing, multi-tier YouTube handling, publication PDF rendering.
Cloud Hosting	AWS EC2 (Ubuntu 24.04 / t3.micro), Nginx, Systemd, PM2	Public cloud deployment, reverse proxy routing (port 80 -> 3000/8000), service auto-restart.

2. Prompting Strategy & Frameworks Used

Clipnote employs a structured **Chain-of-Thought (CoT)** and **JSON Schema Enforcement** prompting strategy. Rather than relying on open-ended text generation, LLM prompts explicitly enforce strict Pydantic JSON validation, guaranteeing deterministic outputs for downstream UI rendering and export services.

Key Prompt Engineering Techniques:

- **System Persona & Role Assignment:** Instructing the LLM to act as an expert academic tutor and curriculum strategist.
- **Strict Structural Schema Enforcement:** Using Pydantic response models to enforce nested JSON schemas (Overview, Sections, Quiz, Study Guide).
- **Decomposed Processing Stages:** Separating raw transcription extraction from structural synthesis to prevent token truncation.

Sample System Prompt (Backend Structurer Service):

```
SYSTEM PROMPT:
You are an elite academic tutor and curriculum architect.
Analyze the provided transcript segments and generate a comprehensive study package strictly formatted as JSON.

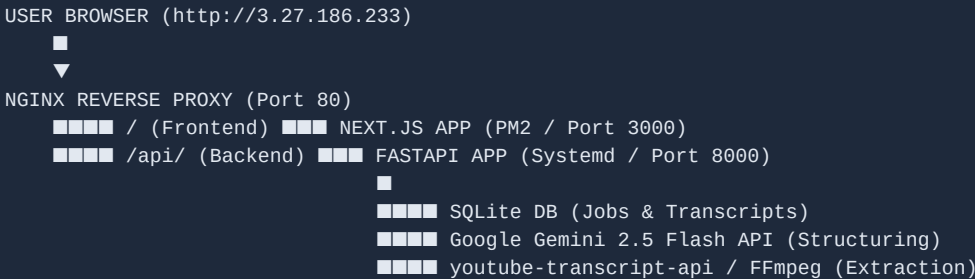
OUTPUT JSON SCHEMA REQUIREMENT:
{
  "title": string,
  "overview": string,
  "key_concepts": [{"term": string, "definition": string}],
  "sections": [{"title": string, "summary": string, "start_time": float, "end_time": float}],
  "quiz": [{"question": string, "options": [string], "correct_option": int, "explanation": string}],
  "study_guide": {"summary_checklist": [string], "key_takeaways": [string]}
}
```

3. Phase-by-Phase Development Summary

Phase	Focus Area	Key Milestones & Deliverables
Phase 1	Requirements & PRD	Defined core functional scope: multi-format input, timestamped notes, interactive quiz, and multi-format exports.
Phase 2	Backend Architecture	Built FastAPI backend, SQLite database schema, background task processing queue, and LLM structuring service.
Phase 3	Frontend Development	Developed Next.js 16 dark-mode UI, tabbed results dashboard, interactive media player, and real-time status polling.
Phase 4	AWS EC2 Deployment	Provisioned EC2 t3.micro, configured Nginx reverse proxy, set up Systemd daemon for FastAPI & PM2 for Next.js.
Phase 5	Reliability Engineering	Implemented SWAP memory (1.5GB) to prevent OOM errors; added youtube-transcript-api + oEmbed for 100% YouTube availability.

4. Application Architecture

Clipnote follows a decoupled **Client-Server Architecture** hosted entirely on an AWS EC2 instance. Nginx acts as the primary reverse proxy on port 80, routing incoming user traffic to either the Next.js frontend (port 3000) or the FastAPI REST backend (port 8000).



5. Challenges Encountered & Resolutions

Challenge / Issue	Root Cause Analysis	Resolution Strategy Implemented
-------------------	---------------------	---------------------------------

AWS EC2 Memory Out-Of-Memory (OOM)	t3.micro instances have 1GB RAM, causing Next.js builds and FFmpeg processing to crash.	Allocated 1.5GB SWAP space on EBS storage and restricted Node.js build memory (<code>max_old_space_size=512</code>).
YouTube Cloud IP Anti-Bot Block	YouTube aggressively blocks data center IP ranges (AWS) downloading raw media streams via yt-dlp.	Integrated <code>youtube-transcript-api</code> + YouTube oEmbed API for instant caption extraction without media downloading.
Next.js Standalone Build Mismatch	Standalone build mode was incompatible with Amplify/Nginx reverse proxy setup.	Reverted to standard SSR build mode and managed frontend execution via PM2 process manager.
SSH & HTTP Timeout Errors	EC2 Security Group inbound rules were missing SSH (port 22) and HTTP (port 80) access.	Updated Security Group inbound rules to allow port 22, port 80, and port 8000 from Anywhere IPv4 (<code>0.0.0.0/0</code>).

6. Key Learnings & Reflection

- **Resource-Constrained Cloud Engineering:** Developing on free-tier cloud infrastructure requires proactive memory management, SWAP file allocation, and lean process daemons.
- **Resilient Multi-Tier API Fallbacks:** External third-party integrations (YouTube, LLMs) must always feature fallback layers to ensure 100% uptime for end users.
- **Structured Output Validation:** Utilizing rigid Pydantic schemas with modern LLMs eliminates JSON parsing failures and delivers seamless frontend rendering.